Kurs Django — Lekcja 5

Panel administracyjny

Cel lekcji

Student potrafi zarejestrować model w panelu admina, dostosować jego wygląd (ModelAdmin) i używa admina do zarządzania danymi.

Blok 1: Teoria (10 min)

Czym jest panel admina

Wbudowana w Django aplikacja (django.contrib.admin), która na podstawie modeli automatycznie generuje interfejs CRUD. Znajduje się już w INSTALLED_APPS od Lekcji 1, a tabele dla niej powstały przy pierwszym migrate.

Do czego służy, a do czego nie

- Świetny do zarządzania danymi przez twórcę/redaktora, szybkiego testowania modeli i wprowadzania danych testowych
- Nie jest przeznaczony jako interfejs dla zwykłych użytkowników strony (oni dostaną własne widoki i formularze w kolejnych lekcjach)

Blok 2: Ćwiczenia w terminalu

Krok 1 — sprawdzenie, czy migracje są zastosowane

```
python manage.py migrate
```

Bez tej wbudowanej tabeli użytkowników createsuperuser zwróci błąd. To dobra okazja, żeby przypomnieć rutynę migracji.

Krok 2 — utworzenie superużytkownika

```
python manage.py createsuperuser
```

Django zapyta o nazwę użytkownika, e-mail (opcjonalny) i hasło. Podczas wpisywania hasła znaki nie są wyświetlane. Warto ostrzec studentów, żeby się nie przestraszyli, że nic się nie dzieje. Django odrzuci zbyt proste hasło (można to potwierdzić, wpisując y, ale tylko na środowisku deweloperskim).

Krok 3 — pierwsze logowanie

```
python manage.py runserver
```

Wejść na http://127.0.0.1:8000/admin/ i zalogować się. Na razie widoczne są tylko Groups i Users. Nasz model Post się nie pojawia, bo jeszcze go nie zarejestrowaliśmy.

Krok 4 — rejestracja modelu

W blog/admin.py:

```
from django.contrib import admin
```

```
from .models import Post

admin.site.register(Post)
```

Odświeżyć stronę admina. Pojawiła się sekcja Posts. Przećwiczyć dodawanie, edycję i usuwanie postów bezpośrednio z panelu.

Krok 5 — dostosowanie widoku listy (ModelAdmin)

Zamieniamy prostą rejestrację na klasę ModelAdmin:

```
from django.contrib import admin
from .models import Post

@admin.register(Post)
class PostAdmin(admin.ModelAdmin):
    list_display = ('tytul', 'data_publikacji', 'opublikowany')
    list_filter = ('opublikowany', 'data_publikacji')
    search_fields = ('tytul', 'tresc')
    ordering = ('-data_publikacji',)
```

(Jeśli studenci dodali pole autor w zadaniu z Lekcji 4, mogą dodać je do list_display i list_filter.)

- list_display — kolumny na liście obiektów
- list_filter — filtry w panelu bocznym
- search_fields — pole wyszukiwania (przeszukuje wskazane pola)
- ordering — domyślne sortowanie (minus = malejąco)

Krok 6 — edycja wprost z listy i akcje

Dodać do PostAdmin:

```
    list_editable = ('opublikowany',)
    list_display_links = ('tytul',)

    actions = ['opublikuj', 'ukryj']

    @admin.action(description="Oznacz wybrane posty jako opublikowane")
    def opublikuj(self, request, queryset):
        queryset.update(opublikowany=True)

    @admin.action(description="Oznacz wybrane posty jako ukryte")
    def ukryj(self, request, queryset):
        queryset.update(opublikowany=False)
```

Przetestować: zaznaczyć kilka postów na liście, wybrać akcję z rozwijanej listy.

Krok 7 — polska wersja panelu i ładniejsze nazwy

W config/settings.py:

```
LANGUAGE_CODE = 'pl'
TIME_ZONE = 'Europe/Warsaw'
```

W blog/models.py, w klasie Post:

```
    class Meta:
        verbose_name = "Post"
```

```
        verbose_name_plural = "Posty"
        ordering = ['-data publikacji']
```

Zmiana Meta zmienia stan modelu w oczach Django, więc ponownie rutyna dwuetapowa:

```
python manage.py makemigrations
python manage.py migrate
```

Odświeżyć admina. Interfejs jest po polsku, a sekcja nazywa się „Posty”.

Blok 3: Zadanie dla studentów

- Dodać do bazy przez panel admina co najmniej 5 postów, w tym 2 nieopublikowane
- Użyć wyszukiwarki i filtrów, żeby znaleźć konkretny post
- Napisać własną akcję w PostAdmin, np. „Dodaj do tytułu prefiks [ARCHIWUM]” (podpowiedź: iteracja po queryset i .save())
- Dodać drugiego użytkownika przez panel (Users → Dodaj) i sprawdzić, co widzi po zalogowaniu, gdy nie ma statusu „staff” i uprawnień do modelu Post

Checklist nawyków

☐ createsuperuser działa dopiero po migrate

☐ Zmiana w models.py (także w Meta!) → makemigrations + migrate

☐ Nowy model → pamiętać o rejestracji w admin.py

☐ Nie instalowaliśmy nowych pakietów, więc pip freeze tym razem niepotrzebny, ale nawyk warto przypomnieć